

OPERATING SYSTEM

Complete Notes — Gate Smashers Playlist

92 Videos • 8 Lessons • Exam & Interview Ready

Source	Gate Smashers — YouTube Playlist
Topics	Intro • Scheduling • Sync • Deadlock • Memory • Disk • File System • Security
Format	Lesson → Video → Notes → Key Concepts → Formulae → Exam Tips
Use-case	B.Tech Exam, GATE, UGC-NET, Technical Interviews

TABLE OF CONTENTS

LESSON 0	Syllabus Overview
LESSON 1	Introduction to Operating System (L1.1 – L1.12)
LESSON 2	CPU Scheduling (L2.1 – L2.11)
LESSON 3	Process Synchronization (L3.1 – L3.13)
LESSON 4	Deadlock (L4.1 – L4.8)
LESSON 5	Memory Management (L5.1 – L5.26)
LESSON 6	Disk Management (L6.1 – L6.10)
LESSON 7	File Systems (L7.1 – L7.7)
LESSON 8	Protection, Security & Extras (L8 + Bonus)

LESSON 0 — Syllabus Overview

Lec-0 OS Syllabus Discussion for College/University & Competitive Exams (GATE, NET)

The Gate Smashers OS playlist covers the full undergraduate + competitive exam syllabus:

SYLLABUS TOPICS COVERED

- ◆ Process Management — scheduling, synchronization, deadlocks
- ◆ Memory Management — paging, segmentation, virtual memory, page replacement
- ◆ Storage Management — disk scheduling, file systems
- ◆ Protection & Security
- ◆ Special Topics — System calls, fork(), threads, linker & loader

EXAM / INTERVIEW MUST-KNOW

- ★ OS is one of the highest-weightage subjects in GATE CSE (~10–12 marks) and UGC-NET.
- ★ Topics like CPU scheduling numericals, Banker's algorithm, and page replacement appear every year in GATE.
- ★ Interview tip: Always explain with an example — e.g., for deadlock, use dining philosophers.

LESSON 1 — Introduction to Operating System

1.1 Introduction to Operating System and its Functions

An Operating System (OS) is system software that acts as an intermediary between hardware and the user. It manages hardware resources and provides a convenient environment for program execution.

KEY CONCEPTS

- ◆ Process Management — creation, scheduling, termination of processes
- ◆ Memory Management — allocation and deallocation of memory
- ◆ File System Management — organise, store and retrieve files
- ◆ I/O Management — control input/output devices via drivers
- ◆ Security & Protection — access control, authentication
- ◆ Network Management — communication between processes/machines
- ◆ User Interface — CLI (Command Line) or GUI (Graphical User Interface)

EXAM / INTERVIEW MUST-KNOW

- ★ OS is NOT just an interface — it is a resource manager.
- ★ Kernel = core of OS (runs in privileged mode); Shell = interface to kernel.
- ★ Interview Q: 'What is the main function of OS?' → Resource management + abstraction layer.

1.2 Batch Operating System | Types of Operating System

Early computers used Batch OS where jobs with similar requirements were grouped and executed together without user interaction during execution.

KEY CONCEPTS

- ◆ Batch OS — Jobs queued in batches; no user interaction during execution. Punch cards era.
- ◆ Advantages: CPU utilization improved; less setup time between jobs.
- ◆ Disadvantages: Debugging difficult; if one job fails, entire batch may fail; no priority.
- ◆ Job Scheduling in Batch: FIFO order typically used.

FORMULAE / DEFINITIONS

Throughput = Number of jobs completed / Time

Turnaround Time = Completion Time - Submission Time

1.3 Multiprogramming and Multitasking Operating System

Multiprogramming keeps multiple programs in memory simultaneously to maximize CPU utilization. Multitasking (Time-sharing) extends this by rapidly switching CPU among processes.

KEY CONCEPTS

- ◆ Multiprogramming — Multiple programs in RAM; CPU switches to another when one does I/O. No time slicing.
- ◆ Multitasking (Time-sharing) — CPU rapidly switches among processes using a time quantum (slice).
- ◆ Degree of Multiprogramming = Number of processes in memory simultaneously.
- ◆ Context Switch — Saving state of current process and loading state of next process.
- ◆ Response time reduced in multitasking vs multiprogramming.

EXAM / INTERVIEW MUST-KNOW

- ★ Multiprogramming ≠ Multiprocessing. Multiprogramming uses ONE CPU; Multiprocessing uses MULTIPLE CPUs.
- ★ GATE Q: Difference between multiprogramming and multitasking — time quantum is the key differentiator.
- ★ Context switch has overhead — it does NOT execute user instructions during the switch.

1.4 Types of OS — Real Time, Distributed, Clustered, Embedded

KEY CONCEPTS

- ◆ Real-Time OS (RTOS) — Strict timing guarantees. Hard RTOS (miss = failure): airbags, pacemakers. Soft RTOS (miss tolerable): video streaming.
- ◆ Distributed OS — Manages a group of independent computers; appears as single system to user.
- ◆ Clustered OS — Multiple CPUs/systems tightly coupled sharing storage for high availability & performance.
- ◆ Embedded OS — Lightweight OS for embedded devices (microcontrollers); e.g., FreeRTOS, VxWorks.
- ◆ Network OS — Provides networking capabilities; each machine has own OS (e.g., Windows Server).

EXAM / INTERVIEW MUST-KNOW

- ★ Hard RTOS: Deadline must ALWAYS be met (medical, military). Soft RTOS: occasional misses tolerable.
- ★ Interview Q: 'Give examples of RTOS' → ABS braking, missile guidance, cardiac pacemaker.
- ★ Distributed OS vs Network OS: In Distributed OS, location transparency exists; users don't know where data physically resides.

1.5 Process States | Schedulers — Long Term, Short Term, Medium Term

A process transitions through multiple states during its lifetime:

FORMULAE / DEFINITIONS

Process States: New → Ready → Running → Waiting → Terminated

New: Process being created.

Ready: Waiting for CPU (in ready queue).

Running: Instructions being executed.

Waiting/Blocked: Waiting for I/O or event.

Terminated: Process finished execution.

KEY CONCEPTS

- ◆ Long-Term Scheduler (Job Scheduler) — Controls degree of multiprogramming; selects from job pool to ready queue. Runs infrequently.
- ◆ Short-Term Scheduler (CPU Scheduler) — Selects which ready process gets CPU. Runs very frequently (ms).
- ◆ Medium-Term Scheduler — Swaps processes in/out of memory (swapping) to control multiprogramming degree.
- ◆ Swapping — Moving a process from RAM to disk (swap out) and back (swap in) to free memory.

EXAM / INTERVIEW MUST-KNOW

- ★ Short-term scheduler is the most frequent; long-term is least frequent.
- ★ GATE Q: Which scheduler controls degree of multiprogramming? → Long-term scheduler.
- ★ A process can go from Running back to Ready (preemption) or to Waiting (I/O request).

1.6 Important Linux Commands — Must Watch for Competitive Exams

KEY CONCEPTS

- ◆ ps — Display currently running processes. 'ps aux' for detailed view.
- ◆ top / htop — Real-time view of running processes and CPU/memory usage.
- ◆ kill <PID> — Terminate a process by Process ID.
- ◆ ls, cd, pwd — List, change directory, print working directory.
- ◆ chmod — Change file permissions (e.g., chmod 755 file).
- ◆ grep — Search text using pattern. 'grep -r pattern dir' for recursive.
- ◆ df -h — Disk space usage; du -sh — Directory size.
- ◆ free -m — Memory usage in MB.
- ◆ ifconfig / ip addr — Network interface configuration.
- ◆ sudo — Execute command as superuser (root).

1.7 System Calls in Operating System and its Types

System calls are the programming interface between user programs and the OS kernel. They allow user-mode programs to request services from the kernel.

KEY CONCEPTS

- ◆ Process Control — fork(), exec(), exit(), wait()
- ◆ File Management — open(), read(), write(), close(), create(), delete()
- ◆ Device Management — ioctl(), read(), write() for devices

- ◆ Information Maintenance — getpid(), alarm(), sleep()
- ◆ Communication — pipe(), shmget(), socket(), send(), recv()

FORMULAE / DEFINITIONS

System Call Mechanism: User program → Trap/Interrupt → Kernel mode → Execute → Return to user mode
 Mode bit: 0 = Kernel mode (privileged), 1 = User mode (restricted)

EXAM / INTERVIEW MUST-KNOW

- ★ System calls use software interrupt (trap) to switch from user to kernel mode.
- ★ fork() creates a child process that is an exact copy of parent.
- ★ exec() replaces current process image with new program.
- ★ GATE Q: Which system call creates a new process? → fork()

1.8 Fork() System Call with Example

fork() creates a new process (child) by duplicating the calling process (parent). Both parent and child continue execution from the next statement after fork().

FORMULAE / DEFINITIONS

Return value of fork(): -1 = error, 0 = in child process, positive PID = in parent process
 Number of processes after n fork() calls = 2^n
 Example: 3 fork() calls → $2^3 = 8$ processes total

KEY CONCEPTS

- ◆ Child process gets a copy of parent's address space (Copy-on-Write in modern OS).
- ◆ Parent and child have different PIDs but same PPID (parent's PID) for child.
- ◆ exec() after fork() — loads new program; common pattern for process creation.
- ◆ wait() — parent waits for child to finish; prevents zombie processes.
- ◆ Zombie process — Child terminated but parent hasn't called wait() yet.
- ◆ Orphan process — Parent terminated before child; child adopted by init (PID 1).

EXAM / INTERVIEW MUST-KNOW

- ★ GATE CLASSIC: 'int x = fork(); fork(); fork(); — how many processes?' → $2^3 = 8$
- ★ Interview: Difference between fork() and exec(): fork creates copy; exec replaces current process.

1.9 Questions on Fork System Call With Explanation

FORMULAE / DEFINITIONS

If fork() is called inside a loop 'for(i=0;i<n;i++) fork();' → 2^n processes created
 Count processes: Draw a process tree. Each fork doubles existing processes.
 If fork() return is checked (if/else), only specific branches execute certain forks.

EXAM / INTERVIEW MUST-KNOW

- ★ Trick: When `fork()` is in an if-else, count only branches that reach subsequent `fork()` calls.
- ★ GATE frequently asks: total number of child/total processes after a fork-tree.

1.10 User Mode and Kernel Mode in Operating System

KEY CONCEPTS

- ◆ User Mode — Restricted mode; processes run here. Cannot directly access hardware or kernel memory.
- ◆ Kernel Mode — Privileged mode; OS kernel runs here. Full hardware access.
- ◆ Mode Switch — Triggered by: System call, interrupt, exception (trap).
- ◆ Protection Ring — Ring 0 (Kernel), Ring 3 (User). x86 architecture uses 4 rings; OS uses 0 and 3.
- ◆ Why separate modes? — Prevent user programs from corrupting kernel or hardware.

EXAM / INTERVIEW MUST-KNOW

- ★ Mode bit in CPU register: 0=Kernel, 1=User.
- ★ Interrupt/Trap sets mode bit to 0 (switches to kernel mode).
- ★ Interview Q: 'What happens when a user program calls `printf`?' → Library call → system call → kernel mode → back to user mode.

1.11 Process vs Threads in Operating System

KEY CONCEPTS

- ◆ Process — Independent program in execution with its own address space, memory, file handles.
- ◆ Thread — Lightweight unit of CPU execution within a process. Shares code, data, heap with sibling threads.
- ◆ Shared among threads: Code segment, Data segment, Heap, File descriptors.
- ◆ Private per thread: Stack, Registers, Program Counter.
- ◆ Benefits of threads: Faster context switch, lower memory overhead, easy data sharing.
- ◆ Multithread example: Web browser — one thread for UI, one for networking, one for rendering.

FORMULAE / DEFINITIONS

Process Context Switch: Save/restore full address space (expensive).

Thread Context Switch: Save/restore only stack + registers (cheap).

EXAM / INTERVIEW MUST-KNOW

- ★ GATE Q: 'What is shared between threads of the same process?' → Code, data, heap, files.
- ★ Interview: 'Why threads over processes?' → Lower overhead, faster communication (shared memory).
- ★ Race condition risk: Since threads share memory, synchronization is needed.

1.12 User Level vs Kernel Level Threads

KEY CONCEPTS

- ◆ User Level Threads (ULT) — Managed by user-space thread library (no kernel awareness). Fast to create/switch. If one thread blocks, entire process blocks.
- ◆ Kernel Level Threads (KLT) — Managed by OS kernel. True parallelism on multi-core. Slower creation/switch than ULT. If one thread blocks, others can run.
- ◆ Hybrid (Many-to-Many model) — Multiple ULTs mapped to multiple KLTs. Best of both worlds.
- ◆ Models: Many-to-One, One-to-One (Linux default), Many-to-Many.

EXAM / INTERVIEW MUST-KNOW

- ★ Linux uses One-to-One model (each user thread = one kernel thread).
- ★ GATE Q: 'Disadvantage of ULT?' → One blocking call blocks all threads of process.
- ★ Interview: pthreads in POSIX, Java threads, Windows threads are KLT-based.

LESSON 2 — CPU Scheduling

2.1 Process Scheduling Algorithms — Preemption vs Non-Preemption

KEY CONCEPTS

- ◆ CPU Scheduling — Decides which ready process gets the CPU and for how long.
- ◆ Preemptive Scheduling — OS can forcibly take CPU from running process (e.g., SRTF, Round Robin, Priority Preemptive).
- ◆ Non-Preemptive Scheduling — Once CPU given, process keeps it until completion or I/O block (e.g., FCFS, SJF, Non-preemptive Priority).
- ◆ Goals of scheduling: Max CPU utilization, Max throughput, Min turnaround time, Min waiting time, Min response time.

2.2 Arrival, Burst, Completion, Turnaround, Waiting and Response Time

FORMULAE / DEFINITIONS

Arrival Time (AT) — Time when process arrives in ready queue.

Burst Time (BT) — Total CPU time required by process.

Completion Time (CT) — Time when process finishes execution.

Turnaround Time (TAT) = CT - AT

Waiting Time (WT) = TAT - BT

Response Time = Time of first CPU allocation - Arrival Time

Average TAT = Sum of all TAT / Number of processes

CPU Utilization = (CPU busy time / Total time) × 100%

Throughput = Number of processes completed / Total time

EXAM / INTERVIEW MUST-KNOW

- ★ GATE always asks: Compute average waiting time / turnaround time — draw Gantt chart first!
- ★ Response time matters for interactive systems; TAT for batch systems.

2.3 First Come First Serve (FCFS) CPU Scheduling

KEY CONCEPTS

- ◆ Non-preemptive. Processes served in order of arrival.
- ◆ Simple to implement using FIFO queue.
- ◆ Problem — Convoy Effect: Short processes wait behind long processes.
- ◆ Not suitable for interactive/time-sharing systems.
- ◆ Example: P1(BT=24), P2(BT=3), P3(BT=3) — WT for P2 = 24, P3 = 27. Avg WT = 17.

EXAM / INTERVIEW MUST-KNOW

- ★ Convoy effect = small processes stuck behind large ones = poor performance.
- ★ FCFS gives maximum average waiting time in most cases — not optimal.

2.4 Shortest Job First (SJF) Scheduling Algorithm

KEY CONCEPTS

- ◆ Non-preemptive. Selects process with smallest burst time from ready queue.
- ◆ Optimal — Gives minimum average waiting time for a given set of processes.
- ◆ Problem — Starvation: Long processes may never get CPU if short ones keep arriving.
- ◆ Solution to starvation: Aging — gradually increase priority of waiting processes.
- ◆ Problem — Burst time must be known in advance (not practical for real systems).
- ◆ Prediction of next burst: Exponential averaging ($\tau(n+1) = \alpha \times t(n) + (1-\alpha) \times \tau(n)$)

EXAM / INTERVIEW MUST-KNOW

- ★ SJF is provably optimal for minimizing average waiting time (non-preemptive).
- ★ GATE Q: 'Which algorithm gives minimum average waiting time?' → SJF.
- ★ Starvation in SJF solved by: Aging or switching to SRTF with aging.

2.5 Shortest Remaining Time First (SRTF) — SJF with Preemption

KEY CONCEPTS

- ◆ Preemptive version of SJF. When new process arrives, compare its burst with remaining burst of current.
- ◆ If new process has shorter remaining time → preempt current, execute new.
- ◆ Gives minimum average waiting time among all scheduling algorithms.
- ◆ Still suffers from starvation for longer processes.

FORMULAE / DEFINITIONS

At each arrival event, check: if new_BT < remaining_BT of current → preempt

WT = TAT - BT (same formula, but need Gantt chart for CT)

EXAM / INTERVIEW MUST-KNOW

- ★ SRTF = most optimal but impractical (requires knowing future burst times).
- ★ GATE numericals: draw Gantt chart at each arrival/completion event.

2.6 Question on SJF with Preemption (SRTF) Scheduling

FORMULAE / DEFINITIONS

Step-by-step approach: (1) At each event (arrival/completion), pick process with min remaining time. (2) Draw Gantt chart. (3) Compute CT for each, then TAT and WT.

Event-driven simulation: sort events (arrivals + completions) on timeline.

2.7 Round Robin (RR) CPU Scheduling Algorithm

KEY CONCEPTS

- ◆ Preemptive. Each process gets a fixed time quantum (q). After quantum expires → back to end of queue.
- ◆ Fair — every process gets CPU in turn; no starvation.
- ◆ Best for time-sharing / interactive systems.
- ◆ Performance depends on quantum size (q):
 - ◆ → Very large q → behaves like FCFS
 - ◆ → Very small q → high context switch overhead
- ◆ Typical quantum: 10–100 ms in practice.
- ◆ Context switches = major overhead in RR.

FORMULAE / DEFINITIONS

If all processes have same BT= b and n processes, each gets CPU every $n \times q$ ms.

Response time $\leq (n-1) \times q$ for a process.

EXAM / INTERVIEW MUST-KNOW

- ★ GATE Q: 'RR with quantum $q=4$, processes P1(5), P2(3), P3(7) — draw Gantt chart and find avg WT.'
- ★ RR gives best response time; SJF/SRTF gives best avg WT.
- ★ Interview: Why is RR used in OS schedulers? → Fairness and low response time.

2.8 Preemptive Priority Scheduling Algorithm

KEY CONCEPTS

- ◆ Each process has a priority number. Lower number = higher priority (or vice versa — check convention).
- ◆ Preemptive: If higher-priority process arrives, current process is preempted.
- ◆ Non-preemptive version: Process runs to completion once selected.
- ◆ Problem: Starvation of low-priority processes.
- ◆ Solution: Aging — Increase priority of a process with each unit of waiting time.
- ◆ Real use: Linux uses nice values (-20 to +19) as priority.

2.9 Example of Mixed Burst Time (CPU & I/O) in CPU Scheduling

KEY CONCEPTS

- ◆ Real processes alternate between CPU burst and I/O burst.
- ◆ I/O-bound processes: many short CPU bursts. CPU-bound processes: few long CPU bursts.
- ◆ During I/O burst, CPU is free to run other processes — key to multiprogramming.

◆ Process goes: Running → Waiting (I/O) → Ready → Running...

FORMULAE / DEFINITIONS

CPU Utilization with multiprogramming: If $P(n)$ = probability all n processes do I/O, then utilization = $1 - P(n)$

2.10 Multi Level Queue Scheduling

KEY CONCEPTS

- ◆ Ready queue split into multiple queues based on process type/priority.
- ◆ Example: Queue 1 = System processes (highest priority), Queue 2 = Interactive, Queue 3 = Batch.
- ◆ Each queue has its own scheduling algorithm (e.g., Q1: RR, Q3: FCFS).
- ◆ No movement between queues — process stays in its assigned queue permanently.
- ◆ Higher-priority queue always served first → lower queues can starve.

2.11 Multilevel Feedback Queue Scheduling

KEY CONCEPTS

- ◆ Enhancement of MLQ — processes CAN move between queues based on behavior.
- ◆ New process enters highest-priority queue. If it uses full quantum → demoted to lower queue.
- ◆ If waiting too long in lower queue → promoted to higher queue (aging).
- ◆ Most flexible and adaptive scheduling — used in modern OS (Windows, Linux CFS concept).
- ◆ Parameters: Number of queues, scheduling per queue, upgrade/downgrade criteria.

EXAM / INTERVIEW MUST-KNOW

- ★ MLFQ is the most general CPU scheduling algorithm — adaptively handles CPU-bound and I/O-bound.
- ★ Interview Q: 'What scheduling does Linux use?' → CFS (Completely Fair Scheduler) — a fair-share variant, not classic MLFQ, but MLFQ concept is asked in theory.
- ★ GATE may ask to trace a MLFQ with given queue definitions.

LESSON 3 — Process Synchronization

3.1 Process Synchronization | Race Condition | Process Types

KEY CONCEPTS

- ◆ Concurrent processes — Multiple processes running simultaneously, sharing resources.
- ◆ Cooperative processes — Interact and share data (need synchronization).
- ◆ Independent processes — Do not share data (no sync needed).
- ◆ Race Condition — Final result depends on the order/timing of process execution. Leads to inconsistent results.
- ◆ Critical Section — Code segment accessing shared resources. Only ONE process should be in it at a time.

FORMULAE / DEFINITIONS

Example of race condition: counter++ in two threads simultaneously. counter++ = LOAD, ADD, STORE (3 instructions). Interleaving causes lost updates.

3.2 Producer Consumer Problem — Classic Synchronization Problem

KEY CONCEPTS

- ◆ Producer creates items and puts in buffer; Consumer takes items from buffer.
- ◆ Shared bounded buffer of size N.
- ◆ Problem without sync: Producer may overfill buffer; Consumer may read from empty buffer.
- ◆ Need: Mutual exclusion on buffer + coordination (producer waits if full, consumer waits if empty).
- ◆ Also called Bounded Buffer Problem.

3.3 Printer-Spooler Problem

KEY CONCEPTS

- ◆ Multiple processes want to print — only one printer available.
- ◆ Spooler directory holds print jobs. Race condition: two processes may get same slot number.
- ◆ Illustrates why critical section protection is needed for shared data structures.

3.4 Critical Section Problem — Mutual Exclusion, Progress, Bounded Waiting

KEY CONCEPTS

- ◆ Critical Section (CS) — Code that accesses shared resources.
- ◆ Entry Section — Code to request permission to enter CS.
- ◆ Exit Section — Code to release the CS.
- ◆ Remainder Section — Rest of the code.

FORMULAE / DEFINITIONS

THREE requirements for any CS solution:

1. Mutual Exclusion: At most one process in CS at a time.
2. Progress: If no process is in CS and some want to enter, selection can't be postponed indefinitely.
3. Bounded Waiting: Limit on how many times OTHER processes can enter CS after a process has requested entry.

EXAM / INTERVIEW MUST-KNOW

- ★ GATE Q: 'Which is NOT a requirement for CS solution?' — know all 3 and their exact meaning.
- ★ Bounded Waiting prevents starvation.

3.5 Lock Variable in OS | Process Synchronization

KEY CONCEPTS

- ◆ Simple software solution: Use a shared lock variable (0=free, 1=busy).
- ◆ Busy waiting: Process loops checking lock — wastes CPU.
- ◆ Problem: Lock variable itself is a shared variable — reading and setting are not atomic → race condition!
- ◆ Solution must make check-and-set atomic.

3.6 Test and Set Instruction

KEY CONCEPTS

- ◆ Hardware solution. TestAndSet() is an ATOMIC hardware instruction.
- ◆ TestAndSet reads current value of lock and sets it to TRUE in one atomic operation.
- ◆ Satisfies: Mutual Exclusion ✓, Progress ✓, Bounded Waiting ✗ (starvation possible).
- ◆ Busy waiting still present — wastes CPU cycles (spinlock).

FORMULAE / DEFINITIONS

```
boolean TestAndSet(boolean *lock) { boolean old = *lock; *lock = true; return old; }
```

Process waits: while(TestAndSet(&lock)); // spin until lock is false

3.7 Turn Variable | Strict Alteration Method

KEY CONCEPTS

- ◆ Software solution for 2 processes using a shared 'turn' variable.
- ◆ Turn variable indicates whose turn it is to enter CS.
- ◆ Satisfies: Mutual Exclusion ✓, Bounded Waiting ✓.
- ◆ Fails: Progress ✗ — Strict alternation; if P0 doesn't want to enter, P1 must still wait.
- ◆ Also called Dekker's idea (simplified). Peterson's Algorithm fixes the progress issue.

3.8 Semaphores | Wait, Signal | Counting Semaphore

KEY CONCEPTS

- ◆ Semaphore — Integer variable accessed via two atomic operations: wait() and signal().
- ◆ Counting Semaphore — Value ranges over unrestricted domain. Used to control access to resources with multiple instances.
- ◆ Binary Semaphore (Mutex) — Value is 0 or 1. Used for mutual exclusion.

FORMULAE / DEFINITIONS

wait(S) / P(S): if $S > 0$ then $S--$ else block process

signal(S) / V(S): $S++$; if any process blocked on S, wake one up

Semaphore initialized to number of available resources.

If initialized to 1 → acts as mutex (binary semaphore).

EXAM / INTERVIEW MUST-KNOW

- ★ P = Proberen (Dutch: to test/decrease); V = Verhogen (to increment).
- ★ GATE Q: 'Semaphore initialized to 5 with 7 processes doing wait() — how many are blocked?' → $7-5 = 2$ blocked.
- ★ Wait = down = P; Signal = up = V.

3.9 Binary Semaphore

KEY CONCEPTS

- ◆ Binary semaphore (mutex) — values only 0 (locked) or 1 (free).
- ◆ Used to ensure mutual exclusion in critical sections.
- ◆ Initialized to 1.
- ◆ Process wanting to enter CS: wait(mutex); ... CS ...; signal(mutex);
- ◆ Advantage over simple locks: Blocked processes sleep (no busy waiting) in semaphore queues.

3.10 Practice Questions on Binary Semaphore

FORMULAE / DEFINITIONS

Pattern: wait(S) before CS, signal(S) after CS.

For ordering between processes: P1 does signal(sync) after task; P2 does wait(sync) before using result.

Deadlock risk: if two processes each wait for the other's semaphore first.

3.11 Solution of Producer Consumer Problem using Semaphore

FORMULAE / DEFINITIONS

Three semaphores: mutex=1 (mutual exclusion), empty=N (free slots), full=0 (filled slots).

Producer: wait(empty); wait(mutex); add to buffer; signal(mutex); signal(full);

Consumer: wait(full); wait(mutex); remove from buffer; signal(mutex); signal(empty);

N = buffer size. Key: wait(empty/full) for coordination; wait(mutex) for mutual exclusion.

EXAM / INTERVIEW MUST-KNOW

★ GATE Q: 'What is the correct order of wait() calls in Producer-Consumer?' → coordination semaphore BEFORE mutex semaphore, else deadlock.

3.12 Solution of Readers-Writers Problem using Binary Semaphore

KEY CONCEPTS

- ◆ Multiple readers can read simultaneously; only one writer can write exclusively.
- ◆ Problem: Allow concurrent reads, exclusive writes.
- ◆ Semaphores: mutex (protect readCount), wrt (exclusive write access).
- ◆ Reader: wait(mutex); readCount++; if first reader → wait(wrt); signal(mutex); READ; ...cleanup...
- ◆ Writer: wait(wrt); WRITE; signal(wrt);
- ◆ First readers-writers problem: Reader preference — writers may starve.
- ◆ Second version: Writer preference — readers may starve.

3.13 Dining Philosophers Problem and Solution using Semaphore

KEY CONCEPTS

- ◆ 5 philosophers, 5 chopsticks (one between each pair). Philosopher needs TWO chopsticks to eat.
- ◆ Problem: If all pick left chopstick simultaneously → deadlock (circular wait).
- ◆ Solutions: (1) Allow max 4 philosophers to sit at once. (2) Odd philosopher picks left first, even picks right first. (3) Pick both atomically (monitor solution). (4) Resource hierarchy.
- ◆ Semaphore solution: chopstick[5] initialized to 1. But naive solution causes deadlock.

EXAM / INTERVIEW MUST-KNOW

- ★ Dining philosophers is the canonical example of deadlock with circular wait.
- ★ Interview Q: 'How to avoid deadlock in dining philosophers?' → Any of the 4 solutions above.
- ★ GATE Q may ask: 'Which condition for deadlock does dining philosophers violate in correct solution?' → Circular wait eliminated.

LESSON 4 — Deadlock

4.1 Deadlock — Concept, Example, Necessary Conditions

Deadlock is a situation where a set of processes are blocked because each process is holding a resource and waiting for a resource held by another process in the set.

FORMULAE / DEFINITIONS

FOUR Necessary Conditions (Coffman Conditions) — ALL must hold for deadlock:

1. Mutual Exclusion — Resource held in non-shareable mode.
2. Hold and Wait — Process holds resources while waiting for others.
3. No Preemption — Resources cannot be forcibly taken from a process.
4. Circular Wait — Circular chain: P1 waits for P2, P2 waits for P3, ..., Pn waits for P1.

EXAM / INTERVIEW MUST-KNOW

- ★ GATE Q: 'Which condition is both necessary AND sufficient?' → None alone; all 4 must hold.
- ★ To PREVENT deadlock: Eliminate at least one Coffman condition.
- ★ Interview: 'Explain deadlock with example.' → Dining philosophers or two processes each holding one file.

4.2 Resource Allocation Graph — Single Instance

KEY CONCEPTS

- ◆ RAG — Directed graph to represent resource allocation and requests.
- ◆ Nodes: Processes (circles), Resources (rectangles with dots = instances).
- ◆ Assignment edge: Resource → Process (resource held by process).
- ◆ Request edge: Process → Resource (process waiting for resource).
- ◆ Single instance: If RAG has a CYCLE → Deadlock definitely exists.
- ◆ Multi-instance: Cycle is necessary but NOT sufficient for deadlock.

4.3 Multi-Instance Resource Allocation Graph

KEY CONCEPTS

- ◆ Multiple instances per resource type. Dots inside rectangle = number of instances.
- ◆ Cycle in RAG with multi-instance resources: May or may not be deadlock.
- ◆ Use Banker's Algorithm or detection algorithm for multi-instance deadlock detection.
- ◆ Knot in RAG → Deadlock in multi-instance case.

4.4 Deadlock Handling Methods and Deadlock Prevention

KEY CONCEPTS

- ◆ 1. Deadlock Prevention — Ensure at least one Coffman condition never holds.
- ◆ 2. Deadlock Avoidance — Dynamically check if granting resource leads to unsafe state (Banker's Algo).
- ◆ 3. Deadlock Detection & Recovery — Allow deadlock, detect it, then recover.
- ◆ 4. Ignore (Ostrich Algorithm) — Used in practice (Windows, Linux) when deadlock is rare.
- ◆
- ◆ Prevention strategies:
- ◆ Mutual Exclusion: Make resources shareable where possible (e.g., read-only files).
- ◆ Hold and Wait: Require all resources at start (resource-intensive) OR release before requesting new.
- ◆ No Preemption: Allow OS to preempt resources (feasible for CPU/memory, not printers).
- ◆ Circular Wait: Impose total ordering on resource types; always request in order.

4.5 Deadlock Avoidance — Banker's Algorithm

KEY CONCEPTS

- ◆ Banker's Algorithm — Avoidance algorithm for multi-instance resources.
- ◆ Safe State — A state is safe if there exists a sequence where all processes can finish.
- ◆ Unsafe State — Deadlock may occur (not guaranteed, but possible).
- ◆ Algorithm refuses resource if granting leads to unsafe state.

FORMULAE / DEFINITIONS

Data structures: Available[m], Max[n][m], Allocation[n][m], Need[n][m]

$Need[i][j] = Max[i][j] - Allocation[i][j]$

Safety Algorithm: Find process P_i where $Need[i] \leq Available$. Add its Allocation to Available. Repeat.

If all processes finish in some order → safe state; grant request.

Resource-Request Algorithm: Pretend to allocate; run safety check; if safe → grant; else → deny.

EXAM / INTERVIEW MUST-KNOW

- ★ GATE 2018 (repeated concept): Given Allocation, Max, Available matrices — find safe sequence.
- ★ Key: $Need = Max - Allocation$. Process finishes → adds its Allocation back to Available.
- ★ Interview Q: 'What is a safe state?' → State where there exists a sequence to complete all processes.
- ★ Limitation: Processes must declare max resources upfront; no dynamic processes.

4.6 – 4.8 GATE Questions on Banker's Algorithm + Deadlock

FORMULAE / DEFINITIONS

GATE 2018 approach: Compute Need matrix → find safe sequence step by step.

At each step: Pick process where $Need \leq Available$ → execute → release resources.

If no such process exists and not all done → unsafe/deadlock.

EXAM / INTERVIEW MUST-KNOW

- ★ Practice: Draw Need matrix clearly. Work through each step showing Available vector update.
- ★ Common mistake: Forgetting to ADD allocated resources of finished process back to Available.

LESSON 5 — Memory Management

5.1 Memory Management and Degree of Multiprogramming

KEY CONCEPTS

- ◆ Memory Management — Efficient allocation of main memory among processes.
- ◆ Goals: Maximize CPU utilization, isolate processes from each other, support multiprogramming.
- ◆ Degree of multiprogramming increases with more efficient memory use.
- ◆ Memory hierarchy: Registers → Cache → RAM → Secondary Storage.

5.2 Memory Management Techniques — Contiguous and Non-Contiguous

KEY CONCEPTS

- ◆ Contiguous Allocation — Process occupies consecutive memory locations. Simple but causes fragmentation.
- ◆ Non-Contiguous Allocation — Process can be spread across non-adjacent memory locations (Paging, Segmentation).
- ◆ Single Partition — Only one process in memory at a time (early systems).
- ◆ Multiple Partitions — Multiple processes in memory simultaneously.

5.3 Internal Fragmentation | Fixed Size Partitioning

KEY CONCEPTS

- ◆ Fixed (Static) Partitioning — Memory divided into fixed-size partitions at boot time.
- ◆ Internal Fragmentation — Allocated memory > required memory. Wasted space INSIDE partition.
- ◆ Example: Partition size = 10KB, Process needs 7KB → 3KB wasted (internal fragmentation).
- ◆ Number of partitions = max degree of multiprogramming.
- ◆ Problem: Large process may not fit any partition; small processes waste large partitions.

FORMULAE / DEFINITIONS

Internal Fragmentation = Partition Size - Process Size

5.4 Variable Size Partitioning

KEY CONCEPTS

- ◆ Dynamic Partitioning — Partition created exactly as needed for each process. No internal fragmentation.
- ◆ External Fragmentation — Total free memory sufficient but not contiguous. Wasted space OUTSIDE partitions.
- ◆ Compaction — Move all processes to one end to merge free space. Expensive operation.

- ◆ Holes — Free memory blocks scattered throughout memory.

5.5 First Fit, Next Fit, Best Fit, Worst Fit Memory Allocation

KEY CONCEPTS

- ◆ First Fit — Allocate first hole big enough. Fast search, may leave small holes at start.
- ◆ Next Fit — Like First Fit but search starts from last allocated position. Spreads allocation.
- ◆ Best Fit — Allocate smallest hole that fits. Minimizes wasted space per allocation, but slow + tiny leftover holes.
- ◆ Worst Fit — Allocate largest hole. Leaves large leftover holes (theory: good for future large requests).

EXAM / INTERVIEW MUST-KNOW

- ★ First Fit and Best Fit are better than Worst Fit in practice.
- ★ GATE Q: Given holes and process sizes, which allocation fits where? — trace each algorithm.
- ★ Best Fit can cause tiny, unusable holes (more external fragmentation in total).
- ★ First Fit: fastest (stop at first fit). Worst Fit: keeps largest hole available.

5.6 – 5.7 Compaction | Swapping

KEY CONCEPTS

- ◆ Compaction — Shuffle memory contents to place all free memory in one large block. Requires relocation.
- ◆ Swapping — Move entire process from RAM to disk (swap out) and back (swap in). Increases multiprogramming.
- ◆ Swap space — Dedicated disk area for swapped processes.
- ◆ Swapping bottleneck: Disk I/O speed. Context switch time dominated by swap time.

5.8 – 5.9 Need for Paging | What is Paging

KEY CONCEPTS

- ◆ Paging — Non-contiguous memory allocation. Eliminates external fragmentation; small internal fragmentation only.
- ◆ Physical memory divided into fixed-size frames. Logical memory divided into same-size pages.
- ◆ Page size = Frame size (typically 4KB).
- ◆ Page Table — Maps logical page number to physical frame number.
- ◆ Logical Address = Page Number (p) + Page Offset (d)
- ◆ Physical Address = Frame Number × Frame Size + Offset

FORMULAE / DEFINITIONS

Logical Address bits = Page Number bits + Offset bits

Page Number = Logical Address / Page Size

Offset = Logical Address mod Page Size

Physical Address = Frame Number $\times$ Page Size + Offset

Page Table size = Number of pages $\times$ Entry size

Number of pages = Logical Address Space / Page Size

Number of frames = Physical Memory / Frame Size

EXAM / INTERVIEW MUST-KNOW

- ★ GATE frequently: Given logical address space = 2^m , page size = 2^n $\rightarrow$ page number bits = $m-n$, offset bits = n .
- ★ Internal fragmentation in paging: max (page_size - 1) bytes wasted in last page.
- ★ Paging has NO external fragmentation.

5.10 – 5.11 Paging Numerical Examples

FORMULAE / DEFINITIONS

Example: Logical addr space = 32 bits, page size = 4KB (2^{12}). Page# bits = $32-12 = 20$. Frames = 2^{20} .

Given logical address 0x3A98: page# = $0x3A98 / 4096 = 3$ (page 3), offset = $0x3A98 \bmod 4096 = 0xA98$

If page 3 maps to frame 7: Physical address = $7 \times 4096 + 0xA98 = 0x7A98$

5.12 – 5.13 Page Table Entries | 2-Level (Multi-Level) Paging

KEY CONCEPTS

- ◆ Page Table Entry (PTE) contains: Frame number, Valid bit, Dirty/Modified bit, Reference bit, Protection bits.
- ◆ Valid bit: 1 = page in memory, 0 = page not in memory (causes page fault).
- ◆ Dirty bit: 1 = page modified, needs to be written back to disk before replacing.
- ◆ 2-Level Paging — Split page number into two parts to reduce page table size in memory.
- ◆ Outer page table $\rightarrow$ Inner page table $\rightarrow$ Frame number.
- ◆ Used when page table itself is too large to fit contiguously in memory.

FORMULAE / DEFINITIONS

2-level: Logical address = p1 | p2 | d (outer page, inner page, offset)

Memory accesses for 2-level paging: 3 (outer PT + inner PT + data). With TLB: 2 if hit.

5.14 Inverted Paging

KEY CONCEPTS

- ◆ One global page table for entire system (not per process).
- ◆ Entry per physical frame, not per logical page.
- ◆ Entry: (PID, Page Number) $\rightarrow$ Frame Number.

- ♦ Advantage: Reduces page table memory (one entry per frame vs one per page per process).
- ♦ Disadvantage: Lookup is slow (search entire table for (PID, page) match). Use hash for speed.

5.15 Shared Memory and Copy-on-Write

KEY CONCEPTS

- ♦ Shared Pages — Multiple processes can share same physical frames (e.g., shared libraries).
- ♦ Pages marked read-only in page table for sharing.
- ♦ Copy-on-Write (COW) — After fork(), parent and child share pages. Write triggers copy of that page only.
- ♦ COW reduces memory usage — pages copied only when modified.

5.16 What is Thrashing

KEY CONCEPTS

- ♦ Thrashing — OS spends more time paging (swapping pages in/out) than executing user processes.
- ♦ Cause: Too many processes in memory → each has insufficient frames → page fault rate explodes.
- ♦ Effect: CPU utilization drops drastically despite high system activity.
- ♦ Solution: Working Set Model — give each process the number of frames it actually needs (its working set).
- ♦ Working Set = set of pages a process references in a time window (Δ).
- ♦ Also solved by: Reducing degree of multiprogramming, increasing RAM.

5.17 Segmentation vs Paging

KEY CONCEPTS

- ♦ Segmentation — Logical view of memory. Divide program into segments (code, stack, heap, data).
- ♦ Segments have variable sizes (unlike pages). Segment table: Base + Limit for each segment.
- ♦ Logical Address = Segment Number + Offset; check Offset < Limit.
- ♦ Physical Address = Base[segment] + Offset.
- ♦ External fragmentation in segmentation (variable-size segments).
- ♦ Paging vs Segmentation: Paging = OS view (fixed), Segmentation = user/programmer view (logical).
- ♦ Modern OS: Segmented paging — combine both (Intel x86 architecture).

5.18 Segmentation Numerical Example

FORMULAE / DEFINITIONS

Given: Segment table with (Base, Limit) pairs.

Logical address (s, d): check $d < \text{Limit}[s]$, else → segmentation fault.

Physical address = $\text{Base}[s] + d$

Example: Seg 2: Base=4096, Limit=1024. Logical (2, 512) → Physical = 4096 + 512 = 4608. Valid.

Logical (2, 1200) → 1200 > 1024 → Segmentation Fault!

5.19 Virtual Memory | Page Fault | Significance

KEY CONCEPTS

- ◆ Virtual Memory — Allows execution of processes not completely in physical memory.
- ◆ Illusion: Each process has its own large address space regardless of physical RAM.
- ◆ Page Fault — Accessing a page not currently in physical memory (valid bit = 0).
- ◆ Page Fault Handling: Trap → OS finds page on disk → find free frame → load page → update PT → restart instruction.
- ◆ Demand Paging — Pages loaded only when demanded (page fault), not in advance.
- ◆ Advantages: More processes can run, programs can be larger than RAM, less I/O needed initially.

FORMULAE / DEFINITIONS

Effective Access Time (EAT) = $(1-p) \times \text{memory_access_time} + p \times \text{page_fault_service_time}$

p = page fault rate (ideally very close to 0)

EXAM / INTERVIEW MUST-KNOW

- ★ GATE Q: 'Calculate EAT given page fault rate and service times.'
- ★ Interview Q: 'What happens on a page fault?' — trace all steps.

5.20 – 5.21 Translation Lookaside Buffer (TLB)

KEY CONCEPTS

- ◆ TLB — Fast associative cache for page table entries. Stores recent (page→frame) mappings.
- ◆ TLB Hit — Page number found in TLB → fast address translation (1 memory access only).
- ◆ TLB Miss — Not in TLB → access page table in memory → update TLB → 2+ memory accesses.
- ◆ TLB is part of MMU (Memory Management Unit) hardware.

FORMULAE / DEFINITIONS

EAT with TLB = $h \times (t_{\text{TLB}} + t_{\text{mem}}) + (1-h) \times (t_{\text{TLB}} + 2 \times t_{\text{mem}})$

h = TLB hit ratio, t_{TLB} = TLB access time, t_{mem} = memory access time

Simplified: $\text{EAT} = t_{\text{TLB}} + t_{\text{mem}} + (1-h) \times t_{\text{mem}}$

EXAM / INTERVIEW MUST-KNOW

- ★ GATE most-asked TLB question: Calculate EAT given hit ratio and access times.
- ★ Toughest variant: Multi-level paging + TLB. Know formula for 2-level: Hit = 2 accesses, Miss = 3.

5.22 – 5.26 Page Replacement Algorithms

When all frames are full and a page fault occurs, OS must replace a page:

KEY CONCEPTS

- ◆ FIFO — Replace the oldest page (first in). Simple but not always optimal.
- ◆ Belady's Anomaly (FIFO only): Adding more frames can INCREASE page faults! Counterintuitive.
- ◆ Optimal (OPT) — Replace page that will NOT be used for the longest time. Impossible in practice (needs future knowledge). Used as benchmark.
- ◆ LRU (Least Recently Used) — Replace page not used for longest time in the past. Best practical algorithm. No Belady's anomaly.
- ◆ MRU (Most Recently Used) — Replace most recently used page. Counter-intuitive; sometimes useful for specific access patterns.

FORMULAE / DEFINITIONS

Page Fault Rate = Number of page faults / Total page references

Hit Ratio = $1 - \text{Page Fault Rate}$

EXAM / INTERVIEW MUST-KNOW

- ★ GATE Q: Given reference string and number of frames — compute page faults for FIFO/LRU/OPT.
- ★ OPT always gives minimum page faults (lower bound). LRU approximates OPT best.
- ★ Belady's Anomaly: FIFO can have more faults with MORE frames — GATE favorite trick.
- ★ LRU implementation: Counter-based or Stack-based (use doubly linked list + hash map).

LESSON 6 — Disk Management & Disk Scheduling

6.1 Hard Disk Architecture

KEY CONCEPTS

- ◆ Platter — Circular disk coated with magnetic material.
- ◆ Track — Concentric circles on a platter.
- ◆ Sector — Smallest unit of storage on a track (typically 512B or 4KB).
- ◆ Cylinder — Set of tracks at same position on all platters.
- ◆ Read/Write Head — Reads/writes data; one per platter surface.
- ◆ Spindle — Motor that rotates platters (5400, 7200, 10000 RPM).
- ◆ Arm — Moves heads across tracks (seek movement).

6.2 Disk Access Time — Seek, Rotational, Transfer Time

FORMULAE / DEFINITIONS

Disk Access Time = Seek Time + Rotational Latency + Transfer Time

Seek Time — Time to move head to correct track. Dominant component.

Rotational Latency — Time for desired sector to rotate under head. Avg = $(1 / (2 \times \text{RPM})) \times 60\text{s}$

Transfer Time = Data Size / Transfer Rate

Average Seek Time $\approx 1/3 \times$ Maximum Seek Time (approximation)

EXAM / INTERVIEW MUST-KNOW

- ★ GATE Q: Calculate total disk access time given seek time, RPM, data size.
- ★ Seek time minimization is the goal of disk scheduling algorithms.

6.3 – 6.9 Disk Scheduling Algorithms

KEY CONCEPTS

- ◆ FCFS — Service requests in arrival order. Fair but high seek time. No optimization.
- ◆ SSTF (Shortest Seek Time First) — Service nearest track first. Minimizes seek; starvation for far requests.
- ◆ SCAN (Elevator) — Head moves in one direction servicing all requests; reverses at end. No starvation.
- ◆ LOOK — Like SCAN but reverses when no more requests in that direction (doesn't go to end).
- ◆ C-SCAN (Circular SCAN) — Services in one direction only; jumps back to start after reaching end. Uniform wait time.
- ◆ C-LOOK — Like LOOK + Circular. Jumps back to lowest pending request after reaching last request.

FORMULAE / DEFINITIONS

Total head movement = Sum of $|current_position - next_request|$ for each move

EXAM / INTERVIEW MUST-KNOW

- ★ SSTF can starve requests — not fair. SCAN/LOOK are better balanced.
- ★ GATE: Given disk queue and initial head position, compute total seek distance for each algorithm.
- ★ C-SCAN gives more uniform wait time than SCAN (no bias toward center tracks).
- ★ Interview: 'Which disk scheduling is used in practice?' → Modified LOOK/C-LOOK variants.

6.10 RAID — Redundant Array of Independent Disks

KEY CONCEPTS

- ◆ RAID 0 — Striping only. No redundancy. Best performance, no fault tolerance.
- ◆ RAID 1 — Mirroring. Exact copy on 2 disks. Fault tolerant, 50% storage efficiency.
- ◆ RAID 5 — Striping with distributed parity. Tolerates 1 disk failure. Minimum 3 disks.
- ◆ RAID 6 — Striping with double parity. Tolerates 2 disk failures. Minimum 4 disks.
- ◆ RAID 10 — Mirror + Stripe. Combines RAID 1 and RAID 0. High performance + redundancy.

LESSON 7 — File System

7.1 File System in Operating System

KEY CONCEPTS

- ◆ File System — Way of organizing and storing files on storage devices.
- ◆ Common file systems: FAT32, NTFS (Windows), ext4 (Linux), APFS (macOS), UFS (Unix).
- ◆ File — Named collection of related information stored on secondary storage.
- ◆ Directory — Container for files and subdirectories. Organizes files into hierarchy.

7.2 File Attributes & Operations

KEY CONCEPTS

- ◆ File Attributes: Name, Type, Location (device + address), Size, Protection (permissions), Timestamps (created/modified/accessed), Owner.
- ◆ File Operations: Create, Write, Read, Seek (reposition), Delete, Truncate.
- ◆ Open file table — OS maintains info about all open files per process.
- ◆ File types: Regular, Directory, Symbolic link, Device file, Socket, Pipe.

7.3 – 7.4 File Allocation Methods — Contiguous vs Non-Contiguous

KEY CONCEPTS

- ◆ Contiguous Allocation — File occupies consecutive disk blocks.
- ◆ Advantages: Simple, fast sequential and direct (random) access.
- ◆ Disadvantages: External fragmentation, file size must be known upfront, difficult to extend.
- ◆ Non-Contiguous Allocation: Linked and Indexed (no external fragmentation).

7.5 Linked List Allocation

KEY CONCEPTS

- ◆ Each block has a pointer to next block. Directory entry: first block address.
- ◆ Advantages: No external fragmentation, file can grow dynamically.
- ◆ Disadvantages: No direct access (must traverse from start), pointer overhead, reliability (lost pointer = lost file).
- ◆ FAT (File Allocation Table) — Modified linked list. Pointers stored in separate table (FAT), not in blocks. Enables faster traversal.

7.6 Indexed File Allocation

KEY CONCEPTS

- ◆ Index block — Dedicated block holding all pointers to file's data blocks.
- ◆ Directory entry: index block address.
- ◆ Supports direct access: index block[i] → data block directly.
- ◆ Advantages: No external fragmentation, supports random access.
- ◆ Disadvantages: Index block overhead; large files need multi-level indexing.

7.7 Unix Inode Structure with Numerical Example

KEY CONCEPTS

- ◆ Inode (Index Node) — Unix data structure storing file metadata and block pointers.
- ◆ Inode contains: File size, permissions, timestamps, block pointers.
- ◆ Pointer structure in Inode:
- ◆ Direct blocks — Point directly to data blocks (e.g., 12 direct pointers).
- ◆ Single Indirect — Points to block containing pointers to data blocks.
- ◆ Double Indirect — Points to block of blocks of data pointers.
- ◆ Triple Indirect — Three levels of indirection for very large files.

FORMULAE / DEFINITIONS

Block size = B bytes, Block address = 4 bytes, Pointers per block = B/4

Direct access: $12 \times B$ bytes

Single indirect: $(B/4) \times B$ bytes

Double indirect: $(B/4)^2 \times B$ bytes

Triple indirect: $(B/4)^3 \times B$ bytes

Max file size = sum of all above

Example: B=1KB, addr=4B → ptrs/block=256. Double indirect = $256^2 \times 1KB = 64MB$

EXAM / INTERVIEW MUST-KNOW

- ★ GATE Q: 'Given inode structure, what is max file size?' — Apply formula above.
- ★ Inode does NOT store filename — filename stored in directory entry pointing to inode.
- ★ Hard link: two directory entries pointing to same inode. Soft link: separate inode pointing to path.

LESSON 8 — Protection, Security & Special Topics

Lec-8 Protection & Security in Operating System

KEY CONCEPTS

- ◆ Protection — Mechanism to control access of processes/users to resources. Internal OS concern.
- ◆ Security — Defense against external threats (unauthorized access, malicious programs). Broader concept.
- ◆ Goals: Confidentiality, Integrity, Availability (CIA triad).
- ◆ Access Matrix — Rows = Domains (users/processes), Columns = Objects (files/devices). Cell = access rights.
- ◆ Access Control List (ACL) — Store access matrix column-wise (per object).
- ◆ Capability List — Store access matrix row-wise (per domain/user).
- ◆ Authentication — Who are you? (password, biometrics, tokens).
- ◆ Authorization — What are you allowed to do? (ACL, RBAC).
- ◆ Threats: Viruses, Worms, Trojans, Ransomware, Rootkits, DDoS.

EXAM / INTERVIEW MUST-KNOW

- ★ Protection = mechanism (HOW to enforce). Security policy = WHAT to enforce.
- ★ Interview Q: 'Difference between ACL and Capability list?' → ACL: per-object; Capability: per-user.
- ★ Unix permissions: rwx for owner/group/others. `chmod 755 = rwxr-xr-x`.

Bonus TLB Toughest Question (GATE Level)

FORMULAE / DEFINITIONS

Multi-level paging + TLB EAT (n-level paging):

TLB Hit: $EAT = t_{TLB} + t_{mem}$

TLB Miss (k-level): $EAT = t_{TLB} + (k+1) \times t_{mem}$

Overall EAT = $h(t_{TLB} + t_{mem}) + (1-h)(t_{TLB} + (k+1) \times t_{mem})$

Simplify: $EAT = t_{TLB} + t_{mem} + (1-h) \times k \times t_{mem}$

Top 15 Top 15 OS Interview Questions — Placement Strategy

TOP 15 INTERVIEW QUESTIONS

- ★ Q1: What is an OS? → Resource manager + abstraction layer between hardware and user.
- ★ Q2: What is a process vs thread? → Process = independent, own address space. Thread = lightweight, shares memory.
- ★ Q3: What is deadlock? → Circular wait for resources. 4 Coffman conditions.

- ★ Q4: What is virtual memory? → Technique to run programs larger than physical RAM using paging.
- ★ Q5: What is thrashing? → High page-fault rate, OS spends more time paging than executing.
- ★ Q6: Difference between paging and segmentation? → Paging: fixed size, no external frag. Segmentation: variable, logical units.
- ★ Q7: What is a semaphore? → Integer variable with atomic wait() and signal() for synchronization.
- ★ Q8: What is context switching? → Saving and restoring process state for CPU sharing.
- ★ Q9: What is the critical section problem? → Ensure mutual exclusion, progress, bounded waiting.
- ★ Q10: What is Banker's Algorithm? → Deadlock avoidance; check if resource grant leads to safe state.
- ★ Q11: Difference between preemptive and non-preemptive scheduling? → CPU can/cannot be taken forcibly.
- ★ Q12: What is TLB? → Hardware cache for page table entries; reduces memory access time.
- ★ Q13: What is a page fault? → Accessing page not in RAM; OS loads from disk.
- ★ Q14: What is Belady's Anomaly? → More frames → more page faults in FIFO. (Does not occur in LRU/OPT.)
- ★ Q15: What is an inode? → Unix structure storing file metadata and block pointers.

Lec-92 Linker & Loader

KEY CONCEPTS

- ◆ Linker — Combines object files into a single executable. Resolves symbolic references between modules.
- ◆ Static Linking — Libraries embedded into executable at compile time. Larger executable but no runtime dependency.
- ◆ Dynamic Linking — Libraries loaded at runtime (DLL/.so files). Smaller executable, shared across programs.
- ◆ Loader — Loads executable from disk into memory for execution.
- ◆ Absolute Loader — Loads program at fixed address.
- ◆ Relocating Loader — Loads at any available address; adjusts addresses accordingly.
- ◆ Bootstrap Loader — First code executed after power-on; loads OS into memory.
- ◆ Loading steps: Allocate memory → Load code/data → Resolve addresses → Transfer control to main().

EXAM / INTERVIEW MUST-KNOW

- ★ Static vs Dynamic linking: Static = bigger executable, no dependency. Dynamic = smaller, runtime dependency.
- ★ Loader vs Linker: Linker combines object files; Loader puts executable in memory.
- ★ Interview Q: 'What is a DLL?' → Dynamic Link Library — shared library loaded at runtime.

QUICK REFERENCE — Key Algorithms at a Glance

Algorithm / Concept	Type	Key Property
---------------------	------	--------------

FCFS Scheduling	CPU	Non-preemptive; convoy effect; simple
SJF	CPU	Min avg WT; starvation; non-preemptive
SRTF	CPU	Preemptive SJF; globally optimal avg WT
Round Robin	CPU	Preemptive; fair; time quantum; no starvation
Priority	CPU	Can be preemptive; starvation → aging fix
MLFQ	CPU	Adaptive; most flexible; modern OS
Banker's Algorithm	Deadlock	Safe state check; avoidance; needs max declared
FIFO Page Replace.	Memory	Belady's anomaly; simple; not optimal
LRU	Memory	Best practical; no Belady's; costly to implement
OPT	Memory	Optimal (min faults); impractical (needs future)
FCFS Disk	Disk	Fair; high seek distance
SSTF	Disk	Min seek; starvation of far requests
SCAN/LOOK	Disk	Elevator; fair; no starvation
C-SCAN/C-LOOK	Disk	Uniform wait; circular; most practical

Notes compiled from Gate Smashers OS Playlist (92 videos) — for academic and interview preparation.